

SALARY PREDICTION APPLICATION

Week 1 Assignment

End-to-End ML Pipeline — From Data to Deployment

■ DEADLINE

Friday — 06:00 AM

THE MISSION

Build and deploy a **Salary Prediction Application**.

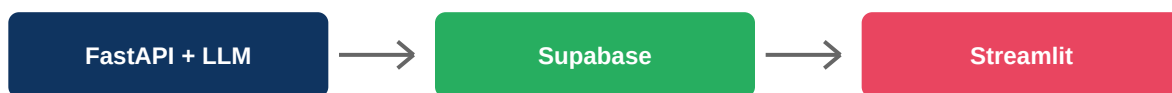
Your app takes in data science job details — experience level, employment type, company size, job title, and more — and predicts the expected salary using a trained Decision Tree model served through an API you build and deploy yourself.

Then, the predicted results are passed to a **local LLM** (via Ollama) that acts as a data analyst — it generates a written narrative with insights and at least one visualization (chart or graph) explaining the salary landscape.

Results are stored in **Supabase** and presented on a live **Streamlit dashboard** your users can interact with.

ARCHITECTURE

This project follows a **pre-generation architecture**. You have a local pipeline and a deployed consumption layer. How they connect and what sits between them is for you to figure out.



Local pipeline → Cloud storage → Deployed dashboard

The deployed FastAPI is a **separate deliverable**. It is not part of the local pipeline flow above. Think of it as a standalone deployment exercise — same model, independently accessible.

DATASET

WHAT YOU NEED TO BUILD

- 1 Clean the Dataset**
Prepare the data for training. Handle what needs handling.
- 2 Train a Decision Tree Model**
Follow best practices in folder structure.
- 3 Build a GET API for Predictions**
All inputs must be validated.
- 4 Python Script That Calls the API**
No unhandled errors. Figure out how to cover the input space.
- 5 LLM-Powered Analysis**
Use a local LLM. It should generate narrative insights and at least one chart or graph. Scoring criteria: storytelling.
- 6 Persist Results to Supabase**
Design your own schema. The dashboard depends on it.
- 7 Streamlit Dashboard**
Display predictions, LLM analysis, and visualizations. The dashboard consumes from Supabase only. Handle missing data gracefully.

THINK ABOUT

How many input combinations do you need? What happens when data is missing? How should your schema support what the dashboard needs? What makes an LLM narrative actually useful vs. generic filler?

These are your problems to solve. No hints.

DELIVERABLES

- URL of the live Streamlit web app
- URL of the deployed FastAPI endpoint (standalone — not part of the local pipeline)
- README file (well presented)

RULES

■ NO GITHUB DESKTOP

Git CLI only. git add, git commit, git push. No shortcuts.

■ NO VIBE CODING

Understand every line you write. You will be asked about it.

Ship it.

Friday, 06:00 AM. No excuses.